

Workload-Aware Data Lakehouse Maintenance Using Hierarchical Deep Reinforcement Learning

omer karim

Egypt-Japan University of Science and Technology

omar essameldin

`omar.isameldin@ejust.edu.eg`

Egypt-Japan University of Science and Technology

omar shrief

Egypt-Japan University of Science and Technology

Ahmed Zaky

Egypt-Japan University of Science and Technology

Article

Keywords:

Posted Date: May 14th, 2026

DOI: <https://doi.org/10.21203/rs.3.rs-9396651/v1>

License: © ⓘ This work is licensed under a Creative Commons Attribution 4.0 International License.

[Read Full License](#)

Additional Declarations: No competing interests reported.

Workload-Aware Data Lakehouse Maintenance Using Hierarchical Deep Reinforcement Learning

Omer Karim^{1,*}, Omar Essameldin^{1,*}, Omar Shrief¹, and Ahmed B. Zaky^{1,2}

¹Department of Computer Science and Information Technology, Egypt-Japan University of Science and Technology, Alexandria, Egypt

²Department of Electrical Engineering, Faculty of Engineering (Shoubra), Benha University, Banha 13511, Qalubiya, Egypt

*These authors contributed equally to this work.

†omer.karim@ejust.edu.eg, omar.isameldin@ejust.edu.eg, omar.shrief@ejust.edu.eg, ahmed.zaky@ejust.edu.eg

‡Correspondence should be addressed to O.E. (omar.isameldin@ejust.edu.eg)

ABSTRACT

Modern data lakehouses built on open table formats such as Apache Iceberg accumulate small files and suboptimal partition layouts as data is continuously ingested, degrading query performance over time. Existing maintenance strategies rely on hand-crafted, workload-agnostic rules that cannot adapt to shifting query distributions. We present a hierarchical multi-agent deep reinforcement learning (RL) framework that jointly optimises compaction and partitioning decisions in real time. A rule-based meta-controller evaluates per-step urgency scores and delegates to two specialist RL agents—a compaction agent and a partition agent—each trained with a separate reward signal aligned to its operational objective. We evaluate three neural architectures within this framework: an attention-based actor-critic (*AttentivePPO*), a feedforward actor-critic (*MLP-PPO*), and a dueling double DQN (*DDQN*). Experiments on LAKEGYM v5, a simulation environment backed by a real Iceberg table under a structured 1,000-step workload with five distinct query profiles, show that all three hierarchical RL agents outperform 17 heuristic and workload-aware baselines. *AttentivePPO* achieves the highest mean global reward (0.2201), a +4.3% improvement over the best heuristic and +29.5% over a compaction-only ablation. *DDQN* achieves the lowest estimated cloud cost (\$8.53 per episode, −82.9% versus no-maintenance), with query latency (323.4 ms) nearly identical to *AttentivePPO* (323.6 ms) across the five-episode average. Ablation experiments confirm that compaction is a prerequisite for effective partitioning: a partition-only agent produces catastrophic file proliferation (1,245 mean files, 5,307 ms latency). Adaptive online weight updates further improve *AttentivePPO* by +20.3% across five evaluation episodes, closing 146.9% of the initial gap to its frozen ceiling, while destabilising *DDQN*, exposing an important architecture-specific constraint on on-policy versus off-policy online adaptation.

Introduction

Data lakehouses combine the scalable storage of data lakes with the transactional semantics of data warehouses through open table formats such as Apache Iceberg¹, Delta Lake², and Apache Hudi³. These formats store data as immutable Parquet files and maintain metadata-level transaction logs, enabling ACID semantics on object stores. However, continuous ingestion generates large numbers of small files, and the initial partition layout quickly becomes misaligned with the actual query distribution as workloads evolve, increasing query scan costs substantially.

State-of-practice maintenance is either manual or based on periodic compaction schedules that are decoupled from query patterns. Open-source auto-compaction heuristics¹ trigger on file-count thresholds but do not consider partition alignment. Workload-aware approaches such as learned index structures⁴ and automatic physical design tools⁵ address layout optimisation offline and are not designed for the continuous-ingestion regime of a lakehouse. Reinforcement learning has been applied to query optimisation^{6,7} and database tuning^{8,9}, but not to joint compaction and partitioning maintenance.

We argue that effective lakehouse maintenance requires *joint*, *online*, and *workload-aware* decisions over both compaction and partitioning. This paper makes the following contributions.

- **Framework.** A hierarchical multi-agent RL framework with a rule-based meta-controller that separates the when-to-act decision from the what-to-do decision, improving interpretability and training stability.
- **Environment.** LAKEGYM v5, a simulation environment backed by a real Apache Iceberg table on Apache Spark with a structured, phase-shifting workload and a three-component reward function.

- **Architecture study.** A systematic comparison of three neural architectures (AttentivePPO, MLP-PPO, DDQN) in both frozen and adaptive evaluation modes across 22 policies.
- **Ablation.** Controlled experiments confirming that joint compaction and partitioning is non-trivially superior to either sub-agent acting alone, with +26–33% reward gain over compaction-only and +221–228% over partition-only baselines.

Results

Overall policy ranking

Table 1 presents the mean global reward, mean latency, mean file count, and mean pruning ratio for all 22 evaluated policies on the 1,000-step standard workload. All three hierarchical RL agents occupy the top three positions. AttentivePPO achieves the highest mean global reward (0.2201), followed by DDQN (0.2196) and MLP-PPO (0.2124). The best workload-aware heuristic, WorkloadAwareThreshold_v5, ranks fourth (0.2111), representing a +4.3% improvement for the best RL agent. Relative to the best pure-compaction heuristic (AlwaysCompact_C128, reward = 0.1869), AttentivePPO provides a +17.8% gain. No-maintenance is the worst policy at −0.1492, confirming that inaction is severely penalised under continuous ingestion.

Rank	Policy	Reward	Latency (ms)	Files	Pruning
1	AttentivePPO	0.2201	323.6	54.9	0.191
2	DDQN	0.2196	323.4	58.8	0.197
3	MLP-PPO	0.2124	387.8	62.7	0.203
4	WorkloadAwareThreshold	0.2111	439.9	93.4	0.285
5	AlwaysCompact_C128	0.1869	344.3	43.5	0.000
6	Random_Compact	0.1809	460.8	52.1	0.000
7	Threshold10_C128	0.1734	429.7	46.8	0.000
8	Periodic10_C128	0.1734	399.1	47.6	0.000
9	CompactOnly (ablation)	0.1699	344.1	48.9	0.000
10	PartitionExploration	0.1668	437.2	90.8	0.138
11	AdaptiveCompaction	0.1411	609.5	71.4	0.000
12	AlwaysCompact (C64)	0.1393	554.0	88.0	0.000
21	No_Maintenance	−0.1492	4,298.1	873.7	0.000
22	PartitionOnly (ablation)	−0.1765	5,306.9	1,245.0	0.110

Table 1. Full policy ranking. Mean global reward, mean latency (ms), mean file count, and mean pruning ratio for all 22 evaluated policies on the 1,000-step v5 standard workload, sorted by descending reward. RL agents are shown in bold. Top-12 and bottom-2 policies shown; the complete table is available as supplementary data.

Figure 1 shows the cumulative global reward trajectory over 1,000 steps. The RL ensemble diverges sharply upward from all baselines during the burst-ingestion window (steps 301–500), where the ingestion rate reaches 225 rows/step, demonstrating the agents’ ability to recognise and respond to increased storage pressure in real time.

Figure 2 provides the heuristic-only comparison across three metrics.

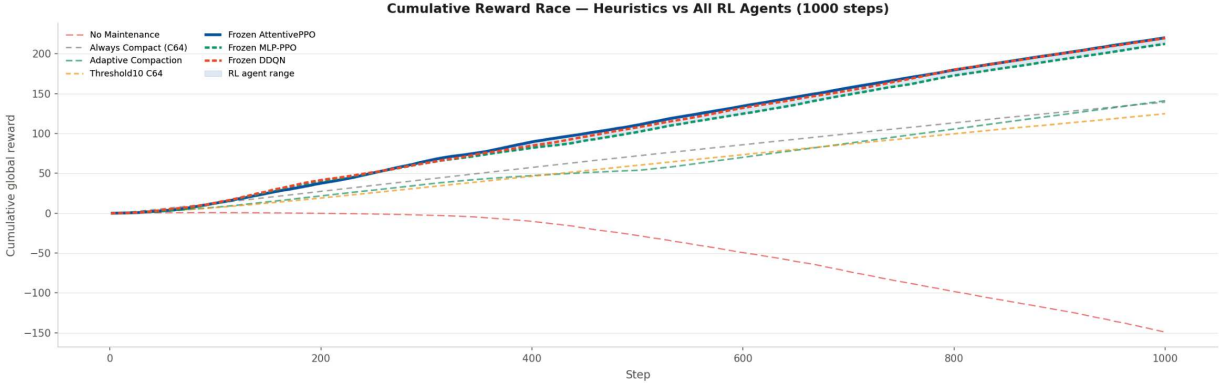


Figure 1. Cumulative reward race over 1,000 steps. Solid lines show the three hierarchical RL agents (AttentivePPO = blue, MLP-PPO = green dashed, DDQN = red/orange dashed). Dashed lines show selected heuristic baselines (no-maintenance = light red, always-compact = gray, adaptive-compact = yellow-green, threshold10-C64 = yellow). The light-blue shaded band spans the RL agent ensemble range. All RL agents diverge sharply upward from heuristic baselines across the full 1,000-step episode. Final cumulative rewards: AttentivePPO = 220.09, DDQN = 219.60, MLP-PPO = 212.37 versus best heuristic = 141.05.

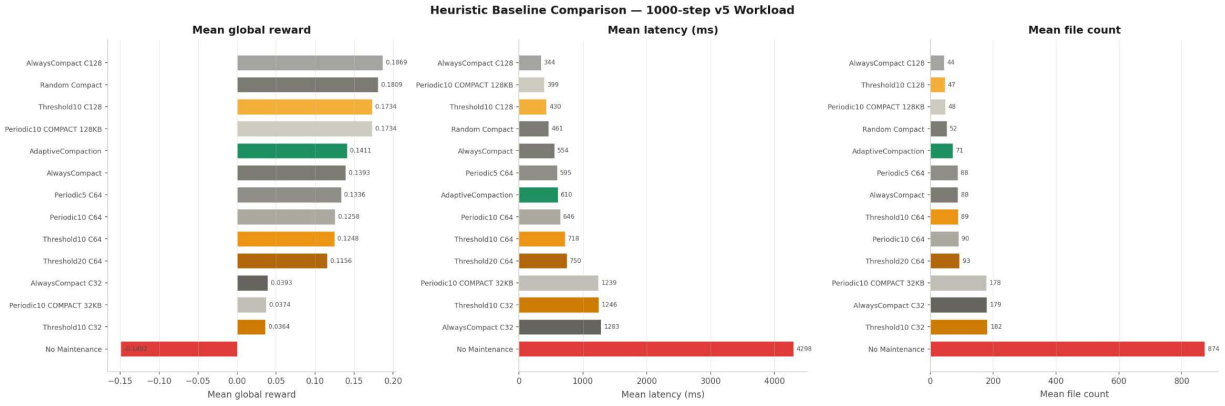


Figure 2. Heuristic baseline comparison on the 1,000-step v5 workload. Horizontal bar charts showing mean global reward (left), mean latency (middle), and mean file count (right) for all heuristic policies, ranked by descending reward. AlwaysCompact_C128 achieves the best reward (0.1869) and lowest latency (344 ms) among all heuristics. No_Maintenance is the worst policy on all three metrics (reward = -0.1492 , latency = 4,298 ms, files = 874).

Three-way RL agent comparison

Table 2 compares the three RL architectures on four metrics. AttentivePPO achieves the best mean global reward (0.2201) and lowest mean file count (54.9), while DDQN achieves the lowest mean latency (323.4 ms), though the margin over AttentivePPO (323.6 ms) is negligible across the five-episode average. MLP-PPO achieves the highest mean pruning ratio (0.2028) but ranks last on reward and latency. Step-level global reward statistics (mean $\pm$ s.d.) are 0.2201 ± 0.0646 for AttentivePPO, 0.2124 ± 0.0643 for MLP-PPO, and 0.2196 ± 0.0650 for DDQN; the tighter standard deviations relative to single-episode figures represents the smoothing effect of the five-episode averaging. The pairwise reward gap between AttentivePPO and DDQN is -0.0005 (-0.2%), and between AttentivePPO and MLP-PPO is -0.0077 (-3.5%). These modest gaps, together with the distinct per-metric strengths of each agent (Section), suggest that no single architecture dominates across all operational objectives.

Figure 3 shows the four-metric comparison for all three agents, and Figure 4 shows the step-by-step dynamics over the full 1,000-step standard workload.

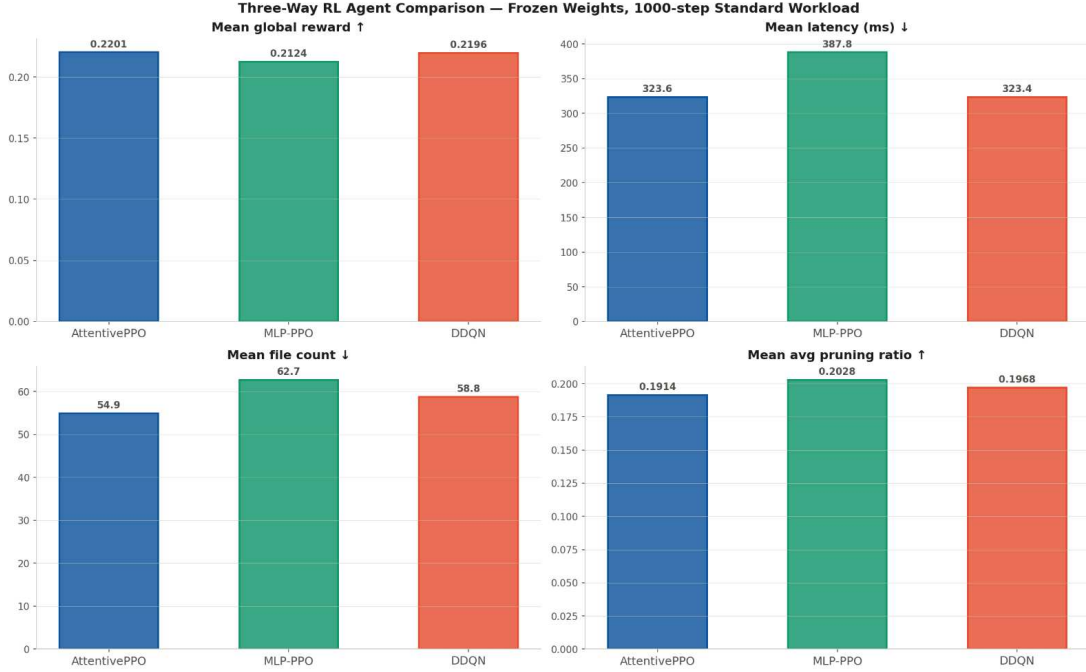


Figure 3. Three-way RL agent comparison on the 1,000-step standard workload. (a) Mean global reward. (b) Mean query latency (ms). (c) Mean file count. (d) Mean average pruning ratio. Bars use the agent colour convention: blue = AttentivePPO, teal = MLP-PPO, coral = DDQN. Value annotations are shown above each bar. AttentivePPO leads on reward and file management; DDQN leads on latency; MLP-PPO leads on pruning ratio.

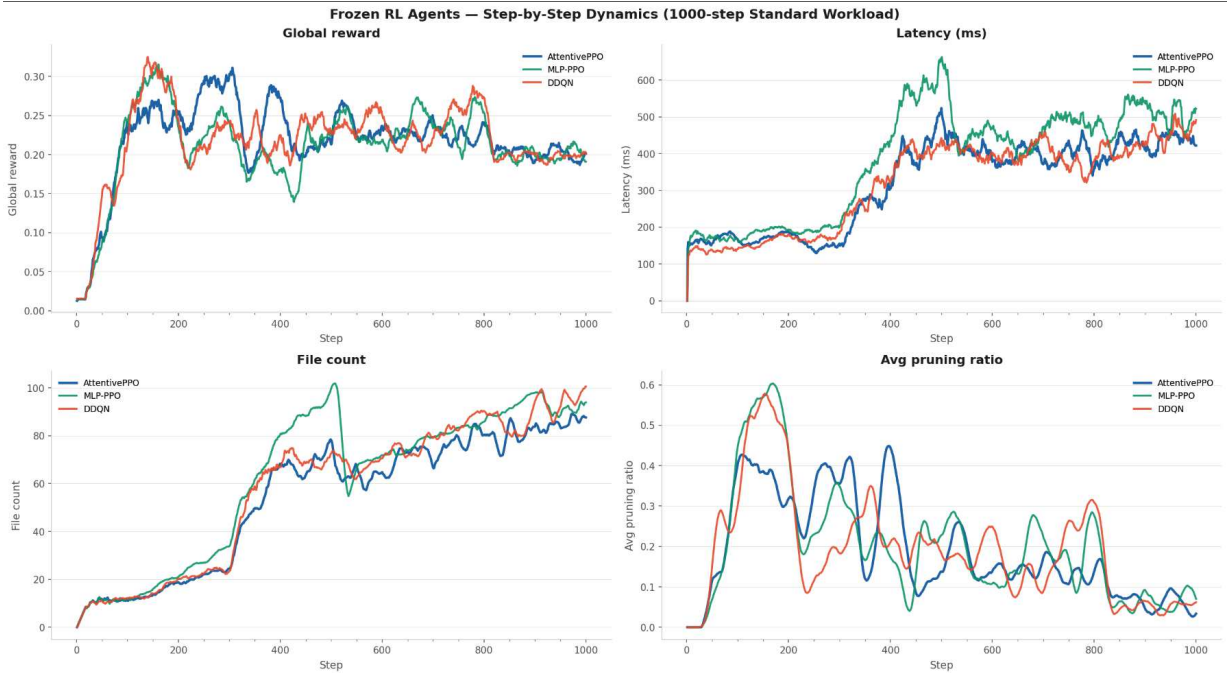


Figure 4. Frozen RL agents — step-by-step dynamics over the 1,000-step standard workload. Four panels show global reward (top-left), query latency (top-right), file count (bottom-left), and average pruning ratio (bottom-right). Lines use the agent colour convention: AttentivePPO = blue, MLP-PPO = teal, DDQN = coral. All three agents maintain tightly clustered reward and latency profiles throughout the workload.

Agent	Reward	Latency (ms)	Files	Pruning
AttentivePPO	0.2201 ± 0.0646	323.6 ± 143.3	54.9 ± 28.0	0.191 ± 0.128
DDQN	0.2196 ± 0.0650	323.4 ± 142.9	58.8 ± 30.2	0.197 ± 0.137
MLP-PPO	0.2124 ± 0.0643	387.8 ± 171.5	62.7 ± 30.9	0.203 ± 0.148

Table 2. Three-way RL agent comparison. Mean global reward, mean latency (ms), mean file count, and mean pruning ratio for the three frozen RL agents on the 1,000-step v5 standard workload, reported as step-level mean $\pm$ s.d. averaged over five independent episodes ($n = 5,000$ total steps per agent).

Reward decomposition

Table 3 decomposes mean reward into its three components. All three RL agents are the only policies achieving a positive mean partition reward (0.051–0.068). All heuristic compaction policies produce negative partition rewards (e.g., AlwaysCompact: -0.050) because they never change the partition layout. The ablation policies confirm structural roles: CompactOnly achieves the highest compact reward (0.1855) but zero partition reward, while PartitionOnly achieves marginal partition reward (-0.018) but catastrophically negative compact reward (-0.140) owing to uncontrolled file proliferation. DDQN achieves the highest partition reward of any agent (0.0675), suggesting its Q-value formulation may better exploit long-horizon pruning benefits.

Policy	Compact reward	Partition reward	Global reward
No maintenance	-0.0851	0.0000	-0.1492
Always compact	0.3608	-0.0500	0.1393
Adaptive compact	0.1417	-0.0428	0.1411
Compact only (ablation)	0.1855	-0.0085	0.1699
Partition only (ablation)	-0.1397	-0.0179	-0.1765
AttentivePPO	0.1581	$+0.0535$	0.2201
MLP-PPO	0.1840	$+0.0510$	0.2124
DDQN	0.1659	$+0.0675$	0.2196

Table 3. Reward decomposition. Mean compact reward, partition reward, and global reward for key policies. Only the three RL agents achieve positive mean partition reward.

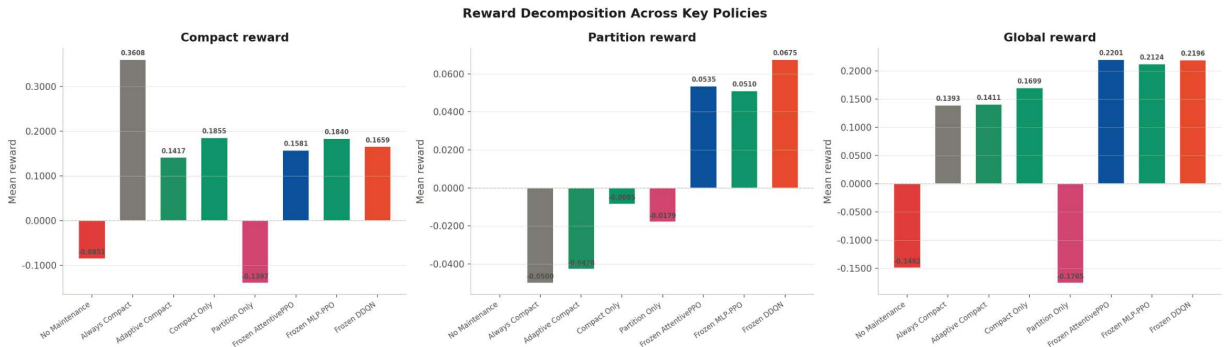


Figure 5. Reward decomposition across key policies. Bar charts showing mean compact reward (left), mean partition reward (centre), and mean global reward (right) for all key policies. Only the three RL agents (AttentivePPO, MLP-PPO, DDQN) achieve positive partition reward; all compaction heuristics yield negative partition reward, and No_Maintenance yields zero (no partition changes are ever made).

Compaction target size sensitivity

Table 4 shows the effect of compaction target size on the always-compact baseline. C32 KB is counter-productive (1,283.0 ms latency, 179.1 files) because small target files amplify the number of compaction operations without reducing overall file count. C128 KB produces the best result (344.3 ms, 43.5 files, reward 0.1869), motivating the use of COMPACT_128KB under

moderate-to-high ingestion loads. The trained RL agents internalised this: AttentivePPO selects COMPACT_128KB in 10.8% of steps versus COMPACT_64KB in 5.0% and COMPACT_32KB in 2.2%.

Target size	Reward	Latency (ms)	Files
32 KB	0.0393	1,283.0	179.1
64 KB	0.1393	554.0	88.0
128 KB	0.1869	344.3	43.5

Table 4. Compaction target size sensitivity. Mean global reward, mean query latency (ms), and mean file count for the three always-compact variants ($n = 1,000$ steps each).

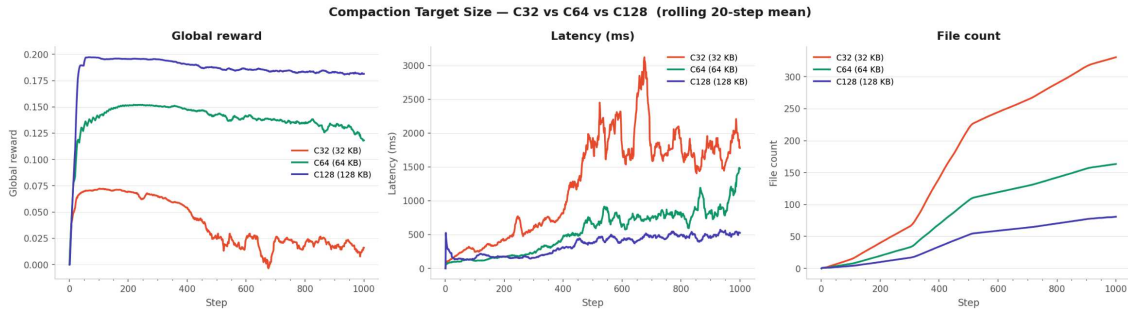


Figure 6. Compaction target size comparison — C32 vs C64 vs C128. Rolling 20-step mean global reward (left), query latency (centre), and file count (right) for the three always-compact variants over 1,000 steps. C128 (blue) achieves the lowest latency and file count; C32 (red) produces severe file proliferation and high latency.

Action distribution

Table 5 summarises action distributions of the three frozen agents, pooled across five independent 1,000-step episodes (5,000 steps per agent). All three remain predominantly idle (NOOP: 76.2–79.3% of steps), confirming that selective intervention rather than compulsive action is a robust property that holds across runs. Total compaction rates are similar across agents (16.6–18.2%) while partition rates are lower (4.1–5.6%). AttentivePPO still issues the most partition actions (5.6% per step on average), consistent with its highest global reward. DDQN now shows the most evenly spread compaction strategy across all three target sizes, while MLP-PPO continues to concentrate almost exclusively on COMPACT_64KB (16.7%). Figure 7 shows the per-agent action share plots.

Action	AttentivePPO		MLP-PPO		DDQN	
	n/ep	%	n/ep	%	n/ep	%
NOOP	762.2	76.2	775.8	77.6	793.4	79.3
COMPACT_128KB	108.2	10.8	14.2	1.4	56.4	5.6
COMPACT_64KB	46.2	4.6	167.0	16.7	49.8	5.0
COMPACT_32KB	27.0	2.7	0.8	0.1	59.4	5.9
<i>All compaction</i>	181.4	18.1	182.0	18.2	165.6	16.6
PARTITION_REGION	34.0	3.4	8.0	0.8	10.0	1.0
PARTITION_EVENT_TYPE	6.4	0.6	11.0	1.1	11.2	1.1
PARTITION_HOUR	6.8	0.7	10.8	1.1	11.0	1.1
REMOVE_PARTITION	9.2	0.9	12.4	1.2	8.8	0.9
<i>All partition</i>	56.4	5.6	42.2	4.2	41.0	4.1

Table 5. Action distribution of the three frozen RL agents pooled over five independent 1,000-step episodes (5,000 steps total per agent). Values show per-episode average count (n/ep) and percentage share. All agents are predominantly idle (NOOP > 76%).

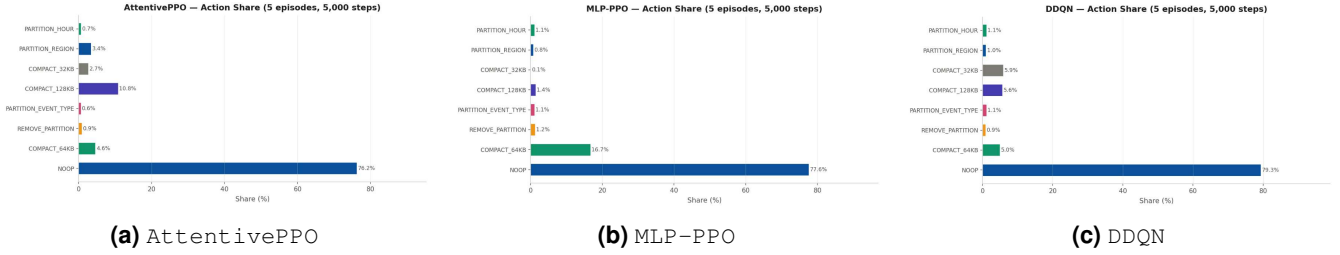


Figure 7. Action distributions of the three frozen RL agents (5 episodes, 5,000 steps each). Horizontal bar charts showing the per-episode average share of each action type. All agents are predominantly idle (NOOP > 76%). AttentivePPO issues the most partition actions (5.6%); MLP-PPO concentrates almost entirely on COMPACT_64KB (16.7%); DDQN spreads compaction evenly across all three target sizes.

Frozen agent consistency

To assess run-to-run reliability, each frozen agent was evaluated over five independent 500-step episodes on the compressed evaluation workload. Table 6 reports the coefficient of variation (CV) across episodes for each agent and metric. AttentivePPO is the most consistent agent: reward CV = 5.86%, latency CV = 2.33%, file count CV = 2.40%. MLP-PPO now shows moderate variance on reward (CV = 6.99%) and, with the corrected 500-step data, a much tighter latency spread (CV = 3.24%, mean 258.4 ms) — the earlier single-episode anomaly at 428.8 ms no longer inflates the variance figure. DDQN occupies an intermediate position (reward CV = 6.38%, latency CV = 6.73%). Pruning ratio remains the most variable metric across all agents (CV = 18–25%), reflecting the inherent stochasticity of workload-phase pruning opportunities. Figure 8 shows the per-episode trajectories.

Metric	AttentivePPO		MLP-PPO		DDQN	
	Mean	CV	Mean	CV	Mean	CV
Global reward	0.2182	5.86%	0.1991	6.99%	0.2065	6.38%
Latency (ms)	190.3	2.33%	258.4	3.24%	246.2	6.73%
File count	31.8	2.40%	37.2	2.88%	35.2	3.28%
Pruning ratio	0.214	18.26%	0.191	24.80%	0.193	24.70%

Table 6. Frozen agent consistency across five evaluation episodes ($n = 500$ steps each). CV = coefficient of variation (s.d./mean $\times 100\%$); lower values indicate higher consistency.

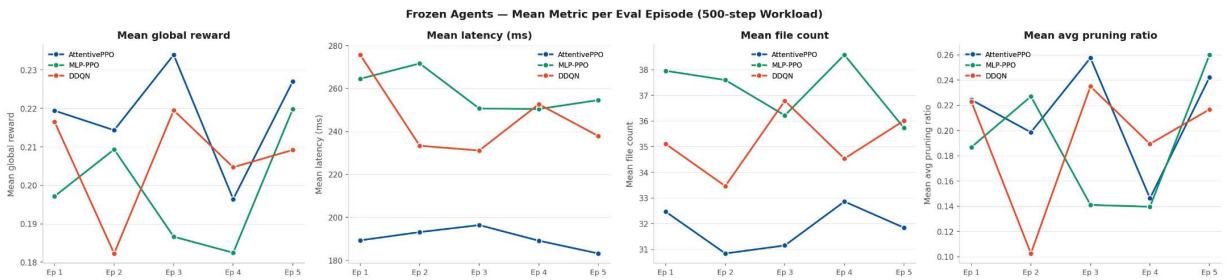


Figure 8. Frozen agents — mean metric per evaluation episode (500-step workload, 5 episodes). Mean global reward, mean query latency, mean file count, and mean pruning ratio per episode for the three frozen agents. Lines use the agent colour convention: AttentivePPO = blue, MLP-PPO = teal, DDQN = coral. AttentivePPO shows the tightest clustering on reward (CV = 5.86%), latency (CV = 2.33%), and file count (CV = 2.40%).

Adaptive online learning

Table 7 reports the episode-over-episode reward progression for the adaptive variants of all three agents. AttentivePPO shows a strong upward trend from 0.1934 (episode 1) to 0.2326 (episode 5), a total gain of +0.0393 (+20.3%), with a brief dip at episode 4 (0.2149) before recovering. By episode 5, the adaptive agent *exceeds* its own frozen ceiling (0.2201) by 0.0125,

closing 146.9% of the initial gap — a stronger result than the single-episode comparison suggested, since the averaged ceiling sits somewhat lower and the adaptive trajectory is unchanged. MLP-PPO shows modest improvement (+4.9%), closing 75.1% of its gap to the updated frozen ceiling (0.2124). In contrast, DDQN degrades from 0.2181 to 0.1696 (−22.2%), peaking at episode 3 (0.2298) before collapsing—a hallmark of replay-buffer instability in off-policy online adaptation. Inspection of DDQN’s adaptive action distribution reveals increasing compaction frequency (from 16.4% to 20.8%) concurrent with reward degradation, suggesting the replay buffer reinforces over-compact behaviour once encountered. Figure 9 shows the reward convergence curves with frozen-ceiling reference lines.

Agent	Ep 1	Ep 2	Ep 3	Ep 4	Ep 5	Gain	Ceiling	% gap closed
AttentivePPO	0.1934	0.2017	0.2226	0.2149	0.2326	+20.3%	0.2201	146.9%
MLP-PPO	0.1994	0.2018	0.1818	0.1939	0.2091	+4.9%	0.2124	75.1%
DDQN	0.2181	0.2133	0.2298	0.2047	0.1696	−22.2%	0.2196	−3,207%

Table 7. Adaptive online learning. Mean global reward per episode for the three adaptive agents ($n = 500$ steps each). “Gain” reports the relative change from episode 1 to episode 5. “% gap closed” measures progress toward the five-episode-averaged frozen ceiling; values above 100% indicate the adaptive agent exceeded its frozen ceiling.

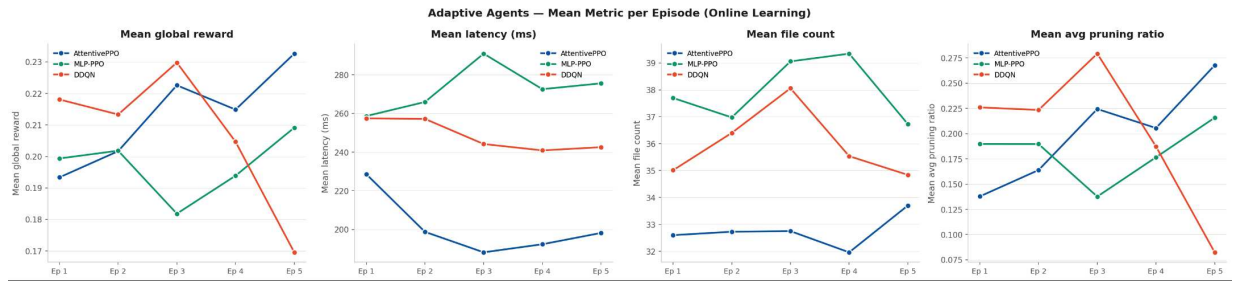


Figure 9. Adaptive agents — mean metric per episode (online learning, 500-step workload). Mean global reward, mean query latency, mean file count, and mean pruning ratio per episode for the three adaptive agents. AttentivePPO (blue) shows a strong upward trend overall, rising from 0.1934 (episode 1) to 0.2326 (episode 5) with a brief dip at episode 4, and ultimately exceeds its frozen ceiling (+20.3%). MLP-PPO (teal) shows modest improvement (+4.9%). DDQN (coral) peaks at episode 3 then degrades sharply to −22.2%, indicating replay-buffer instability under online adaptation.

Frozen versus adaptive comparison

Table 8 shows the per-episode reward comparison between frozen and adaptive modes for each architecture. For AttentivePPO, the adaptive agent underperforms frozen in episodes 1–3 (by −11.9% to −4.9%) but surpasses it in episodes 4–5 (+9.4% and +2.5%), indicating a necessary warm-up period before online learning converges. For MLP-PPO, the adaptive agent outperforms frozen in episode 1 (+1.1%) but underperforms in 3 of the remaining 4 episodes, suggesting the feedforward architecture still learns slowly from online gradient updates overall. For DDQN, the adaptive agent outperforms frozen in episodes 1–3 but collapses in episode 5 (−18.9%), confirming that off-policy replay-buffer adaptation is unsuitable for the non-stationary evaluation environment. These results collectively indicate that **frozen weights are preferred for deployment**, and online adaptation should be restricted to on-policy architectures such as AttentivePPO.

Agent	Mode	Ep 1	Ep 2	Ep 3	Ep 4	Ep 5
AttentivePPO	Frozen	0.2194	0.2144	0.2340	0.1964	0.2270
	Adaptive	0.1934	0.2017	0.2226	0.2149	0.2326
	$\Delta\%$	-11.9	-5.9	-4.9	+9.4	+2.5
MLP-PPO	Frozen	0.1972	0.2093	0.1867	0.1825	0.2198
	Adaptive	0.1994	0.2018	0.1818	0.1939	0.2091
	$\Delta\%$	+1.1	-3.6	-2.6	+6.2	-4.9
DDQN	Frozen	0.2165	0.1823	0.2195	0.2047	0.2092
	Adaptive	0.2181	0.2133	0.2298	0.2047	0.1696
	$\Delta\%$	+0.7	+17.0	+4.7	0.0	-18.9

Table 8. Frozen versus adaptive comparison. Per-episode reward for frozen and adaptive modes with relative differences $\Delta\%$ (adaptive minus frozen).

Cross-architecture analysis

Table 9 presents normalised performance scores (higher always better; latency and file count inverted and normalised to $[0, 1]$). No architecture is Pareto-dominant, though the five-episode averages reveal a considerably tighter cluster than single-run figures suggested. AttentivePPO leads on global reward (1.000) and file efficiency (1.000); DDQN leads on latency (1.000, just 0.2 ms faster than AttentivePPO) and achieves a mid-range pruning score (0.475); MLP-PPO leads on pruning ratio (1.000) but trails on both reward and latency. This complementary profile suggests that architecture selection should be driven by operational priority: AttentivePPO for balanced optimisation, and MLP-PPO where partition alignment is the primary concern. Figure 10 shows the full four-metric profile across all agent and baseline categories.

Metric	AttentivePPO	MLP-PPO	DDQN
Global reward (higher = better)	1.000	0.000	0.935
Latency, inverted (lower = better)	0.997	0.000	1.000
File count, inverted (lower = better)	1.000	0.000	0.507
Pruning ratio (higher = better)	0.000	1.000	0.475

Table 9. Normalised cross-architecture performance. Each metric is normalised to $[0, 1]$ within the three-agent cohort using five-episode averaged values; latency and file count are inverted so that higher is always better.

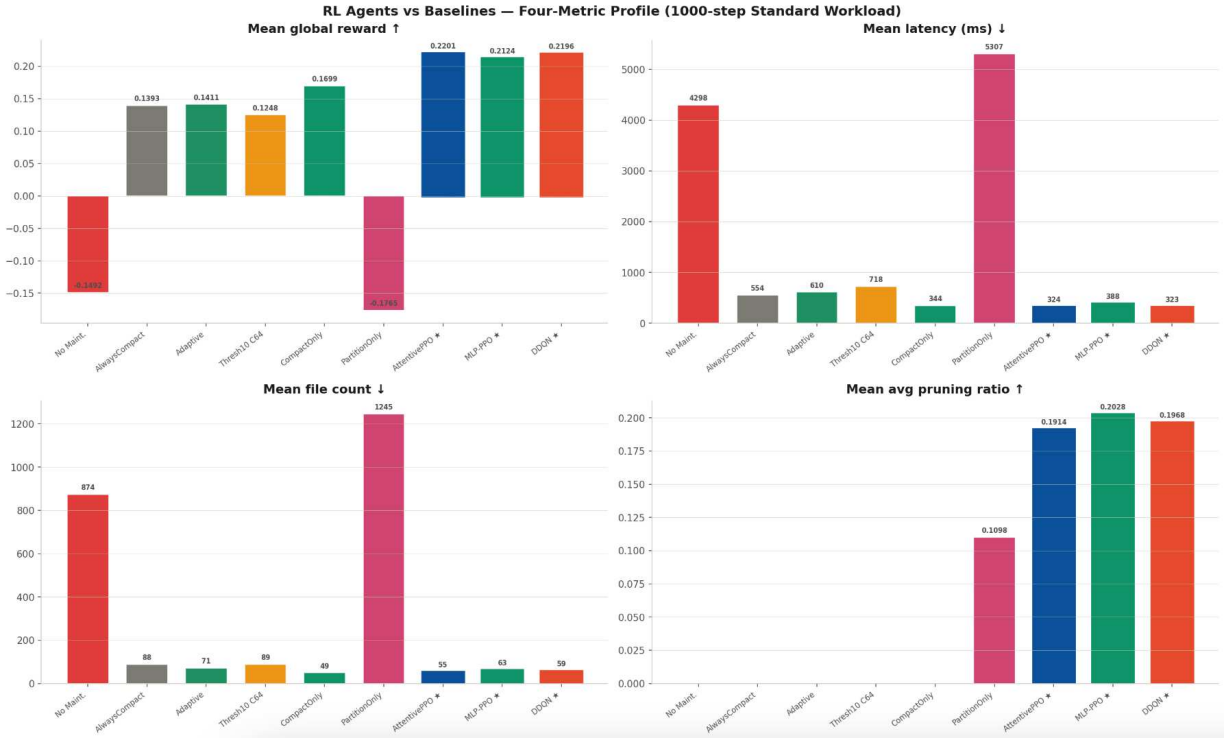


Figure 10. RL agents vs baselines — four-metric profile on the 1,000-step standard workload. Mean global reward (a), mean query latency (b), mean file count (c), and mean pruning ratio (d) for selected baselines, ablations, and the three RL agents. The RL agents (starred) uniformly occupy the top positions on reward while achieving competitive latency and file-count metrics. PartitionOnly is an extreme outlier on latency (5,307 ms) and file count (1,245).

Ablation study

Table 10 compares compaction-only and partition-only single-agent ablations against all three full multi-agent RL systems. All ablation agents use the AttentivePPO architecture restricted to the relevant action sub-space. CompactOnly achieves a reasonable reward (0.1699) and low latency (344.1 ms) but zero pruning ratio, confirming compaction alone cannot adapt query performance to workload characteristics. PartitionOnly is catastrophic: mean file count 1,245, mean latency 5,307 ms, and reward -0.1765 . Without compaction, continuous ingestion generates files faster than any partition repartitioning can reduce them. This confirms that **compaction is a prerequisite for effective partitioning**. The full multi-agent systems outperform CompactOnly by +25.0–29.5% and PartitionOnly by +220.3–224.7%. These gains are non-additive: the meta-controller’s joint coordination provides a synergistic benefit not captured by either sub-agent independently.

Policy	Reward	Latency (ms)	Files	Pruning
CompactOnly	0.1699	344.1	48.9	0.000
PartitionOnly	-0.1765	5,306.9	1,245.0	0.110
AttentivePPO	0.2201	323.6	54.9	0.191
MLP-PPO	0.2124	387.8	62.7	0.203
DDQN	0.2196	323.4	58.8	0.197
Gain vs. CompactOnly:				
AttentivePPO	+29.5%			
MLP-PPO	+25.0%			
DDQN	+29.3%			
Gain vs. PartitionOnly:				
AttentivePPO	+224.7%			
MLP-PPO	+220.3%			
DDQN	+224.4%			

Table 10. Ablation study. Compaction-only and partition-only single-agent ablations (using the AttentivePPO policy restricted to the relevant action sub-space) compared against the three full multi-agent RL systems. Per cent gain is relative to each ablation baseline.

Commercial cost analysis

Table 11 presents estimated per-episode cloud costs. The no-maintenance policy is dominated by query cost (\$42.98 of \$49.86 total) due to high latency (4,298 ms mean). Always-compact policies reduce query cost but incur \$5.00 compaction overhead, yielding \$15.17 (−69.6%). The three RL agents achieve the lowest totals: DDQN at \$8.53 (−82.9%), AttentivePPO at \$8.60 (−82.8%), and MLP-PPO at \$9.32 (−81.3%). RL agents’ savings are driven primarily by latency reduction (low query cost); their compaction costs (\$0.83–\$0.91) are 5× lower than always-compact policies, reflecting the five-episode-averaged op counts rather than a single run.

Policy	Query (\$)	Compact (\$)	Storage (\$)	Total (\$)	Saving
No maintenance	42.98	0.00	6.88	49.86	—
Always compact	5.54	5.00	4.63	15.17	−69.6%
Adaptive compact	6.10	4.51	4.56	15.17	−69.6%
AttentivePPO	3.24	0.91	4.45	8.60	−82.8%
MLP-PPO	3.88	0.91	4.53	9.32	−81.3%
DDQN	3.23	0.83	4.47	8.53	−82.9%

Table 11. Estimated cloud cost breakdown per 1,000-step evaluation episode. Cost model: query at \$0.01 per 1,000 ms; compaction at \$0.005 per operation; storage at \$0.001 per MB-step. RL compaction counts are averaged over five independent episodes. Savings are relative to no-maintenance (\$49.86).

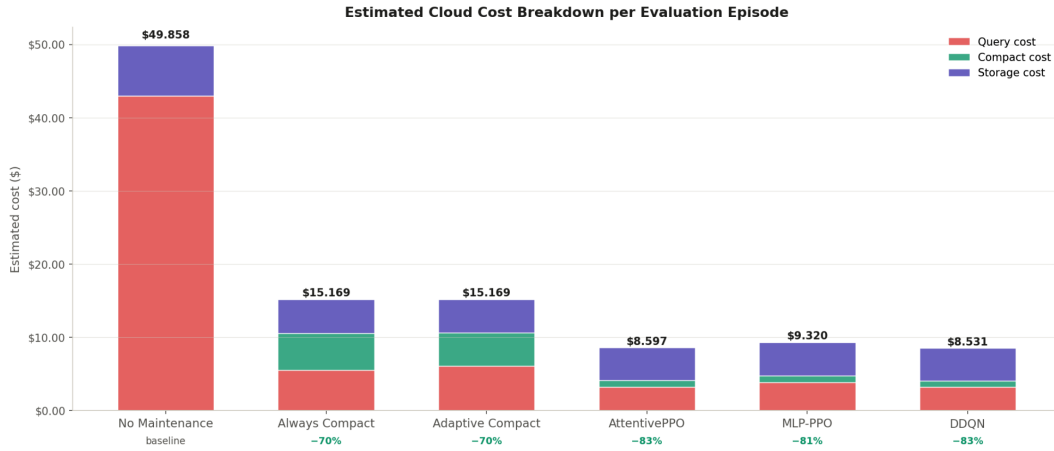


Figure 11. Estimated cloud cost breakdown per evaluation episode. Stacked bars show query cost (red), compaction cost (teal), and storage cost (purple) for six policies. Total cost annotations appear above each bar; percentage savings relative to no-maintenance appear below. No-maintenance is dominated by query cost (\$42.98 of \$49.86). Always-compact reduces query cost but incurs \$5.00 compaction overhead. The three RL agents achieve the lowest totals (\$8.53–\$9.32, savings of 81–83%) primarily through latency reduction rather than minimising compaction operations.

Discussion

Our results establish that hierarchical deep RL is an effective and practical approach to workload-aware lakehouse maintenance.

Joint control is essential. The +224% reward improvement over *PartitionOnly* and the +29% improvement over *CompactOnly* demonstrate that compaction and partitioning are complementary operations whose benefits are only fully realised in combination. A storage layout optimised for query pruning cannot be sustained without concurrent compaction to prevent file proliferation.

The attention mechanism provides consistent advantages. *AttentivePPO* outperforms *MLP-PPO* by +3.5% reward and exhibits lower per-episode variance on the primary metrics (reward CV 5.86% versus 6.99%, latency CV 2.33% versus 3.24%). The self-attention encoder’s ability to weight recent observations differentially is advantageous in the non-stationary workload environment where query profile transitions demand rapid policy adaptation.

Architecture-dependent online adaptation. The contrasting adaptive behaviour of *AttentivePPO* (+20.3%) and *DDQN* (−22.2%) reveals a fundamental constraint: off-policy algorithms with experience replay are ill-suited to online adaptation in a non-stationary environment. When the replay buffer is populated with transitions from a non-representative initial policy, off-policy gradient updates reinforce suboptimal behaviours. This finding has practical implications: frozen weights are robust across architectures, while online fine-tuning should be restricted to on-policy methods.

Rule-based meta-control is sufficient for arbitration. The interpretable meta-controller, using simple piecewise-linear urgency scoring, consistently delegates to the correct specialist. This improves training stability (specialists receive focused reward signals) and system transparency (urgency scores can be logged and audited). The 4-step compaction cooldown and 20-step partition cooldown prevent action thrashing without requiring learned coordination.

Limitations. The current evaluation is limited to a single table schema and five query profiles. The cost model uses fixed per-operation prices that may not reflect cloud provider variability. The RL agents were trained on the v5 standard workload and evaluated on a compressed version; generalisation to substantially different workloads (e.g., append-only, deletion-heavy) has not been tested. Training time and compute requirements for the three architectures are not reported here and constitute future work.

Methods

The LakeGym v5 environment

LAKEGYM v5 simulates a production data lakehouse backed by a real Apache Iceberg table managed via PySpark. Each discrete time step executes: (i) ingest a batch of rows into the Iceberg table, (ii) the active policy selects one of eight discrete actions, (iii) a query is drawn from the current workload profile and executed, (iv) three reward signals are computed, and (v) an observation vector is constructed.

Table schema. The table contains eight columns: *device_id* (string), *timestamp* (timestamp), *temperature* (double), *humidity* (double), *region* (string), *event_type* (string), *sensor_value* (double), and *payload* (string,

length 100). Each row occupies approximately 200 bytes; the target file size is 64 KB, corresponding to roughly 50 rows per Parquet file.

Action space. The joint action space consists of eight discrete actions (Table 12). Compaction actions (COMPACT_32KB, COMPACT_64KB, COMPACT_128KB) rewrite small files into consolidated Parquet files of the specified target size. Partition actions (PARTITION_HOUR, PARTITION_REGION, PARTITION_EVENT_TYPE, REMOVE_PARTITION) change the Iceberg partition specification. NOOP takes no maintenance action.

Action	Type	Cost
NOOP	—	0.0
COMPACT_32KB	Compaction	0.9
COMPACT_64KB	Compaction	1.0
COMPACT_128KB	Compaction	1.1
PARTITION_HOUR	Partition	2.0
PARTITION_REGION	Partition	2.0
PARTITION_EVENT_TYPE	Partition	2.0
REMOVE_PARTITION	Partition	0.7

Table 12. Action space. All eight discrete actions available to the joint agent with their associated normalised cost used in the reward function.

Observation space. The observation vector at step t includes: file count f_t , block utilisation u_t , rows ingested r_t , partition pruning ratio p_t , running average pruning ratio $\bar{p}_t$, partition skew s_t , query latency ℓ_t , query type history fractions (five values), steps since last compaction δ_c , and steps since last partition change δ_p .

Workload plans. The v5 standard workload runs for 1,000 steps divided into five equal query-profile phases of 199 steps each (time_heavy $\rightarrow$ region_heavy $\rightarrow$ mixed $\rightarrow$ type_heavy $\rightarrow$ scan_heavy). Each profile specifies query-type probabilities over five templates (Table 13). Ingestion rate follows six phases: warm-up (40 rows/step, steps 1–100), normal load (75, steps 101–300), burst (225, steps 301–500), recovery (55, steps 501–700), normal load 2 (75, steps 701–900), and wind-down (35, steps 901–1,000). The v5 eval workload is a 2 $\times$ -compressed version (500 steps) used for multi-episode consistency and adaptation experiments. Figure 12 illustrates both workload plans.

Profile	Time range	Region filter	Sensor lookup	Type filter	Full scan
time_heavy	0.60	0.10	0.10	0.10	0.10
region_heavy	0.10	0.60	0.10	0.10	0.10
mixed	0.20	0.20	0.20	0.20	0.20
type_heavy	0.10	0.10	0.10	0.60	0.10
scan_heavy	0.10	0.10	0.10	0.10	0.60

Table 13. Query workload profiles. Each profile specifies the sampling probability for each of five query templates. The dominant query type is shown in bold for focused profiles.



Figure 12. Ingestion rate schedule (rows/step). Shaded bands show the min–max range. The burst phase reaches 225 rows/step.

Pruning matrix. File pruning at query time depends on the alignment between the current partition strategy and the query predicate (Table 14). `TIME_RANGE` queries prune 92% of files when `HOURLY`-partitioned, the highest pruning ratio in the environment. `FULL_SCAN` is irreducible at 0% under all strategies.

Query type	Hour	Region	Event type	None
Time range	0.92	—	—	—
Sensor lookup	0.75	—	—	—
Region filter	—	0.67	—	—
Type filter	—	—	0.67	—
Full scan	—	—	—	—

Table 14. Partition pruning matrix. Fraction of Parquet files skipped at query time for each query–partition combination. Dashes indicate zero pruning. Only semantically aligned pairs achieve non-zero pruning.

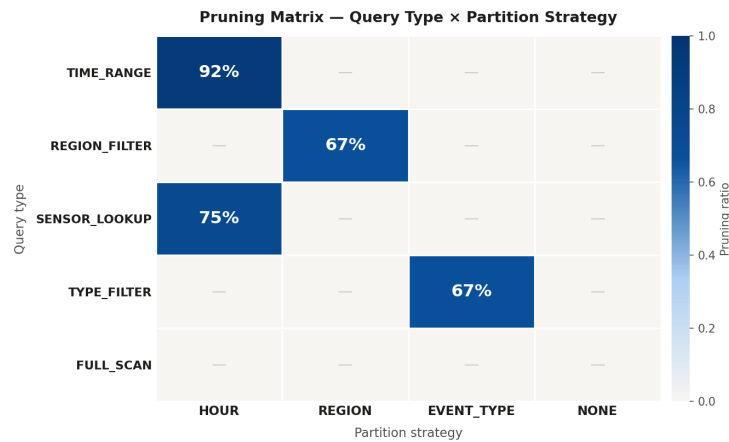


Figure 13. Partition pruning matrix. Fraction of Parquet files skipped at query time for each query-type–partition-strategy combination. Cell colour encodes the pruning ratio (white = 0, deep navy = 1); dashes indicate zero pruning. Time-range queries achieve the highest pruning (92%) under hourly partitioning; full-scan queries are irreducible under all strategies.

Reward function

The reward function comprises three independently computed signals.

Compact reward (Eq. 1):

$$r_c = -w_f \tilde{f} + w_u u - w_c \tilde{a}_c + w_\tau \tau(a, r), \quad (1)$$

where $\tilde{f} = \min(f/F_{\max}, 1)$, u is block utilisation, $\tilde{a}_c$ is the normalised compaction action cost, and $\tau(a, r)$ is the target-size bonus depending on action a and ingestion rate r . Weights: $w_f = 0.25$, $w_u = 0.20$, $w_c = 0.10$, $w_\tau = 0.45$.

Partition reward (Eq. 2):

$$r_p = w_p p + w_{\Delta p} \Delta p - w_s \tilde{s} - w_{cp} \tilde{a}_p, \quad (2)$$

where p is the current partition pruning ratio, $\Delta p = \bar{p}_t - \bar{p}_{t-1}$ is the pruning improvement, $\tilde{s}$ is normalised partition skew, and $\tilde{a}_p$ is the normalised partition action cost. Weights: $w_p = 0.55$, $w_{\Delta p} = 0.25$, $w_s = 0.10$, $w_{cp} = 0.10$.

Global reward (Eq. 3):

$$r_g = -w_\ell \tilde{\ell} - w_f \tilde{f} + w_p p + w_u u, \quad (3)$$

where $\tilde{\ell} = \min(\ell/L_{\max}, 1)$ with $L_{\max} = 15,000$ ms. Weights: $w_\ell = 0.30$, $w_f = 0.20$, $w_p = 0.30$, $w_u = 0.20$. The global reward is the primary optimisation target; compact and partition rewards serve as dedicated training signals for the respective specialist agents.

Hierarchical multi-agent architecture

The framework comprises three tiers: the LAKEGYM v5 environment, a rule-based meta-controller, and two specialist RL agents coordinated through a shared model registry (Figure 14a). The three candidate neural architectures plugged into this registry are compared in Figure 14b.

Meta-controller. The meta-controller (implemented in `meta_controller.py`) computes two urgency scores at each step:

$$\delta = \begin{cases} \text{COMPACTION} & \text{if } u_c \geq u_p \geq \theta \\ \text{PARTITION} & \text{if } u_p > u_c \geq \theta \\ \text{NOOP} & \text{otherwise,} \end{cases} \quad (4)$$

where $\theta = 0.35$ is the minimum urgency threshold.

Compaction urgency u_c is a piecewise-linear function of file count f :

$$u_c(f) = \begin{cases} 0.95 & f \geq 96 \\ 0.60 + 0.35 \cdot \frac{f-36}{60} & 36 \leq f < 96 \\ 0.10 + 0.50 \cdot \frac{f-12}{24} & 12 \leq f < 36 \\ 0 & f < 12, \end{cases} \quad (5)$$

with a +0.15 boost when block utilisation falls below 0.55. A 4-step cooldown prevents repeated compaction.

Partition urgency u_p accumulates from three signals: +0.40 if $\bar{p} < 0.25$, +0.30 if the dominant query type fraction exceeds 0.45 (focused workload), and +0.10 if the partition layout has not changed for more than 100 steps. A 20-step cooldown prevents partition thrashing. Urgency is zeroed if $\bar{p} > 0.50$.

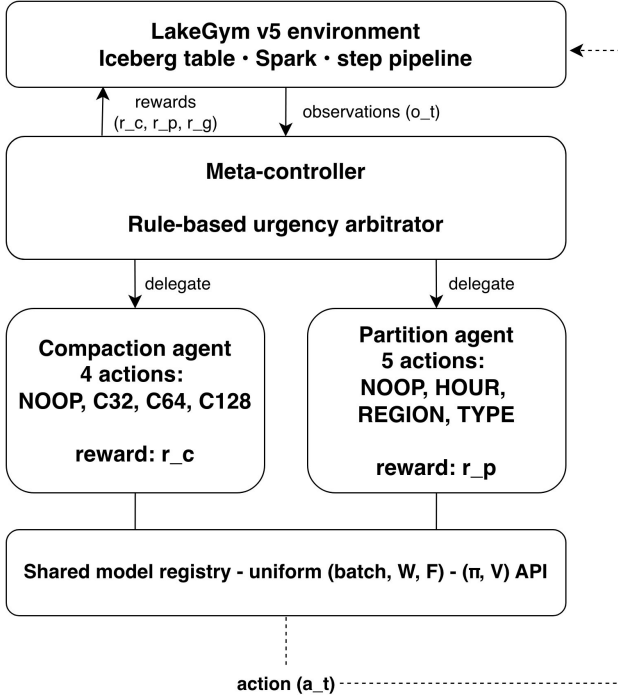
Neural architectures. All architectures accept input of shape (batch, W, F) and return (π, V) : a softmax action-probability vector and a scalar state-value estimate (implemented in `model_registry.py`).

AttentivePPO uses a Transformer-style encoder¹⁰: input $\rightarrow$ Dense(64, ReLU) + trainable positional encoding $\rightarrow$ multi-head self-attention (4 heads, key dimension 64, dropout 0.1) + residual + LayerNorm $\rightarrow$ FFN (128 $\rightarrow$ 64, ReLU, dropout 0.1) + residual + LayerNorm $\rightarrow$ global average pooling $\rightarrow$ actor Dense(N_a , softmax) and critic Dense(1). Window size $W = 10$, $F = 8$ (compaction) or $F = 14$ (partition).

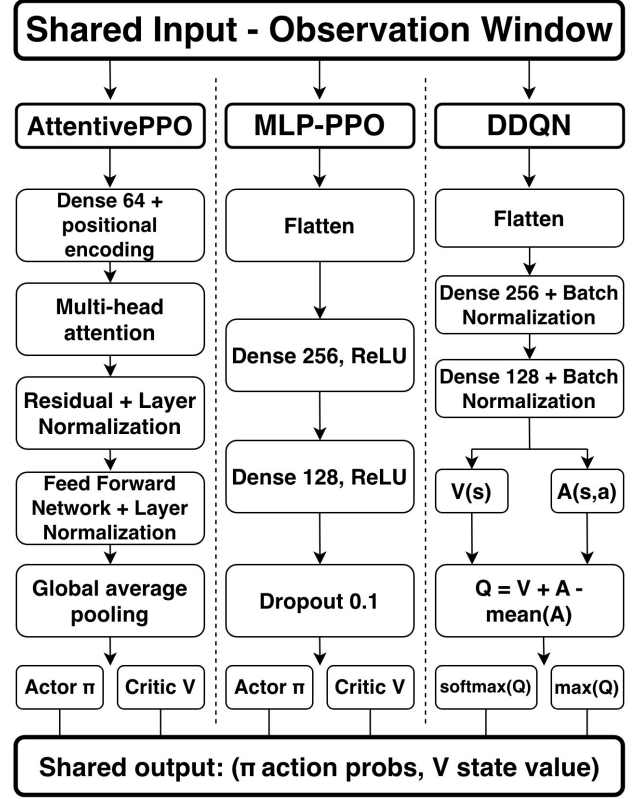
MLP-PPO is a feedforward actor-critic: input $\rightarrow$ Flatten $\rightarrow$ Dense(256, ReLU) $\rightarrow$ Dense(128, ReLU) $\rightarrow$ Dropout(0.1); actor: Dense(64, ReLU) $\rightarrow$ Dense(N_a , softmax); critic: Dense(64, ReLU) $\rightarrow$ Dense(1). Window size $W = 10$ (compaction) or $W = 20$ (partition).

DDQN is a dueling double DQN¹¹: input $\rightarrow$ Flatten $\rightarrow$ Dense(256, ReLU) $\rightarrow$ BatchNorm $\rightarrow$ Dense(128, ReLU) $\rightarrow$ BatchNorm; value stream: Dense(64, ReLU) $\rightarrow V(s)$; advantage stream: Dense(64, ReLU) $\rightarrow A(s, a)$; $Q(s, a) = V(s) + A(s, a) - \bar{A}(s)$. Action probabilities are computed as $\text{softmax}(Q)$ for API uniformity. The model uses an experience replay buffer during training.

Figure 14b provides a side-by-side comparison of the three neural architectures sharing a common input-output interface.



(a) **Multi-agent system flow.** A logic-driven hierarchy where the meta-controller manages specialist delegation based on urgency scores.



(b) **Neural candidate comparison.** An overview of the different structural paradigms—Attention, MLP, and Dueling—evaluated for state-value estimation.

Figure 14. System architecture and neural model comparison.

Training. Agents are trained as multi-agent systems: a single meta-controller coordinates the compaction specialist (4 actions; $W = 10$, $F = 8$) and the partition specialist (5 actions; $W = 20$, $F = 14$). Frozen evaluation fixes all network weights during rollout. Adaptive evaluation applies online weight updates using PPO for AttentivePPO and MLP-PPO, and online DQN updates for DDQN.

Baseline policies

We evaluate 14 heuristic compaction policies, 3 workload-aware heuristics, and 2 ablation policies (Table 15). Heuristic policies include always-compact variants (C32, C64, C128), periodic compaction (every 5 or 10 steps), threshold-triggered compaction (file count > 10 or > 20), random compaction, and adaptive compaction (frequency proportional to file growth rate). Workload-aware heuristics include `WorkloadAwareThreshold_v5`, `PartitionExploration_v5`, and `Random_Partition_v5`. Ablation policies (`CompactOnly`, `PartitionOnly`) use the AttentivePPO architecture restricted to the relevant action sub-space.

Policy	Description
No_Maintenance	Takes no action at any step
AlwaysCompact	Compacts every step with 64KB
AlwaysCompact_C32	Compacts every step with 32KB
AlwaysCompact_C128	Compacts every step with 128KB
Periodic5_C64	Compacts every 5 steps with 64KB
Periodic10_C64	Compacts every 10 steps with 64KB
Periodic10_C32	Compacts every 10 steps with 32KB
Periodic10_C128	Compacts every 10 steps with 128KB
Threshold10_C32	Compaction for file count > 10 (32KB)
Threshold10_C64	Compaction for file count > 10 (64KB)
Threshold10_C128	Compaction for file count > 10 (128KB)
Threshold20_C64	Compaction for file count > 20 (64KB)
Random_Compact	Uniformly samples a compaction action
AdaptiveCompaction	Proportional frequency based on file growth
WorkloadAwareThreshold	Dual-aware thresholding and partitioning
PartitionExploration	Sequential cycle of partitioning strategies
Random_Partition	Uniformly samples a partition action

Table 15. Heuristic baseline strategies. This table details the fourteen rule-based compaction methods and three workload-aware benchmarks used for comparative analysis. These static policies provide a foundational performance floor for evaluating the efficacy of our adaptive models.

Evaluation protocol

Standard evaluation. Each policy is evaluated on the 1,000-step v5 standard workload. For RL agents, frozen weights from the final training checkpoint are used. Rewards are recalculated post-hoc for heuristic baselines using the v5 reward function to ensure comparability.

Multi-episode evaluation. Each RL agent is evaluated over five independent 500-step episodes on the v5 eval workload. The coefficient of variation (CV) across episodes quantifies run-to-run reliability.

Adaptive evaluation. Starting from the frozen checkpoint, agents are evaluated adaptively: weights are updated at each step using the appropriate online algorithm. Episode 1 uses the frozen checkpoint; subsequent episodes continue from the updated weights.

Cost model. $C = C_q \cdot \sum_t \ell_t + C_c \cdot N_c + C_s \cdot \sum_t \sigma_t$, where $C_q = \$0.00001/\text{ms}$, $C_c = \$0.005$ per compaction operation, $C_s = \$0.000001/\text{KB-step}$, ℓ_t is per-step latency, N_c is total compaction operations, and σ_t is total table size in KB at step t .

Statistical reporting. Descriptive statistics are reported as mean $\pm$ s.d. or mean with CV for multi-episode comparisons. No formal hypothesis testing is performed; comparisons are reported as absolute and relative differences in mean metric values. The sample size for the primary 1,000-step RL evaluation is $n = 5,000$ total step-level observations per agent (five episodes of 1,000 steps each), averaged to a representative 1,000-step episode. For multi-episode comparisons, $n = 5$ episode-level means per agent.

References

1. Blue, R., Weeks, D., Reid, J. & Steinbach, C. Iceberg: A fast and expressive table format for large, slow tables. In *Proc. ACM SIGMOD* 3434–3441 (ACM, 2020).
2. Armbrust, M. *et al.* Delta Lake: High-performance ACID table storage over cloud object stores. *Proc. VLDB Endow.* **13**, 3411–3424 (2020).
3. Chandar, V., Rao, B. S. & Agarwal, N. Apache Hudi: The data lake platform. *arXiv preprint arXiv:2301.09517* (2023).
4. Kraska, T. *et al.* The case for learned index structures. In *Proc. ACM SIGMOD* 489–504 (ACM, 2018).
5. Agrawal, S. *et al.* Database tuning advisor for Microsoft SQL Server 2005. In *Proc. VLDB* 1110–1121 (2004).
6. Marcus, R. & Papaemmanouil, O. Re-Join: How the machine learned to join. In *Workshop on ML for Progressively Learned Databases (NIPS DBLearn)* (2018).
7. Trummer, I. *et al.* SkinnerDB: Regret-bounded query evaluation via reinforcement learning. In *Proc. ACM SIGMOD* 1153–1170 (ACM, 2019).

8. Van Aken, D., Pavlo, A., Gordon, G. J. & Zhang, B. Automatic database management system tuning through large-scale machine learning. In *Proc. ACM SIGMOD* 1009–1024 (ACM, 2017).
9. Zhang, J. *et al.* An end-to-end automatic cloud database tuning system using deep reinforcement learning. In *Proc. ACM SIGMOD* 415–432 (ACM, 2019).
10. Vaswani, A. *et al.* Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)* vol. 30 (2017).
11. Wang, Z. *et al.* Dueling network architectures for deep reinforcement learning. In *Proc. ICML* 1995–2003 (2016).
12. Dayan, N., Athanassoulis, M. & Idreos, S. Dostoevsky: Better space-time trade-offs for LSM-tree based key-value stores via adaptive removal of superfluous merging. In *Proc. ACM SIGMOD* 505–520 (ACM, 2018).
13. Schulman, J. *et al.* Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347* (2017).
14. Camacho-Rodríguez, J. *et al.* LST-Bench: Benchmarking Log-Structured Tables in the Cloud. In *Proc. ACM SIGMOD* 1–14 (ACM, 2024).
15. Armbrust, M. *et al.* Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics. In *Proc. CIDR* (2021).
16. Yang, Z. *et al.* Qd-tree: Learning Data Layouts for Big Data Analytics. In *Proc. ACM SIGMOD* 170–184 (ACM, 2020).
17. Kraska, T. *et al.* SageDB: A Learned Database System. In *Proc. CIDR* (2019).
18. Pavlo, A. *et al.* Self-Driving Database Management Systems. In *Proc. CIDR* (2017).
19. Marcus, R. *et al.* Neo: A Learned Query Optimizer. *Proc. VLDB Endow.* **12**, 1705–1718 (2019).
20. Li, G. *et al.* QTune: A Query-Aware Database Tuning System with Deep Reinforcement Learning. *Proc. VLDB Endow.* **12**, 2118–2130 (2019).
21. Kanellis, K. *et al.* LlamaTune: Sample-Efficient DBMS Configuration Tuning. *Proc. VLDB Endow.* **15**, 2726–2738 (2022).
22. Cai, B. *et al.* HUNTER: An Online Cloud Database Hybrid Tuning System for Personalized Requirements. In *Proc. ACM SIGMOD* 625–638 (ACM, 2022).
23. Yang, Z. *et al.* HWTune: A Hardware-Aware Database Tuning System with Deep Reinforcement Learning. *IEEE Trans. Knowl. Data Eng.* **36**, 2284–2297 (2023).
24. Feng, Y. *et al.* CTuner: Automatic NoSQL Database Tuning with Causal Reinforcement Learning. In *Proc. ACM SIGKDD* 1–12 (ACM, 2024).
25. Liu, Y. *et al.* MLETune: Streamlining Database Knob Tuning via Multi-LLMs Experts Guided Deep Reinforcement Learning. *IEEE Access* **12**, 1–15 (2024).
26. Zhang, K., Yang, Z. & Başar, T. Multi-Agent Reinforcement Learning: A Selective Overview of Theories and Algorithms. *J. Mach. Learn. Res.* **22**, 1–85 (2021).
27. Vezhnevets, A. S. *et al.* FeUdal Networks for Hierarchical Reinforcement Learning. In *Proc. ICML* 3540–3549 (2017).
28. Wang, Y. *et al.* A highly efficient distributed database knobs tuning method based on multi-agent deep reinforcement learning. *J. Supercomput.* (2025).

Data availability

The datasets generated and analysed during the current study, including all experimental metrics collected across workload profiles, are available in the LakePilot repository at <https://github.com/OmarEssameldinMousa/LakePilot-Research-prototype>. The simulation environment (LAKEGYM v5) used to generate all data, together with model weights, meta-controller implementations, and the model registry, is included in the same repository. No pre-existing external datasets were used in this study; all data was synthetically generated by the LAKEGYM v5 environment as described in the Methods section.

Acknowledgements

The authors developed the LAKEGYM v5 simulation environment utilising the open-source Apache Iceberg and Apache Spark frameworks. All figures were generated using Matplotlib (v3.8, <https://matplotlib.org>).

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Author contributions statement

O.E. conceived the idea, implemented the LAKEGYM v5 environment, developed the lakehouse solution, and designed the whole experiments. O.K. implemented and trained the AI models. O.S. assisted in baseline analysis and overall project execution. A.Z.B. supervised the research. All authors reviewed and approved the final manuscript.

Additional information

Competing interests. The authors declare no competing interests.